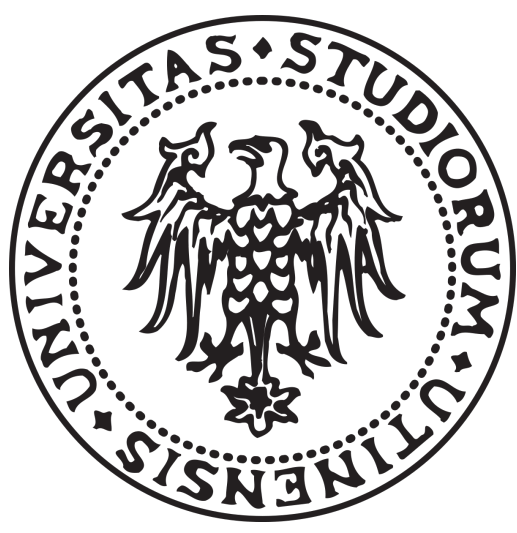




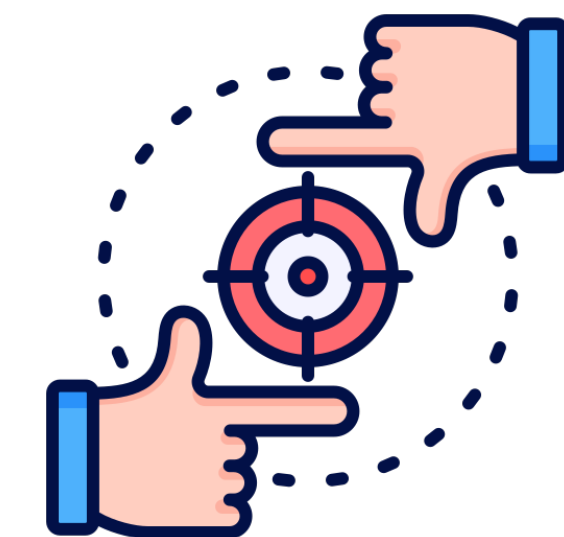
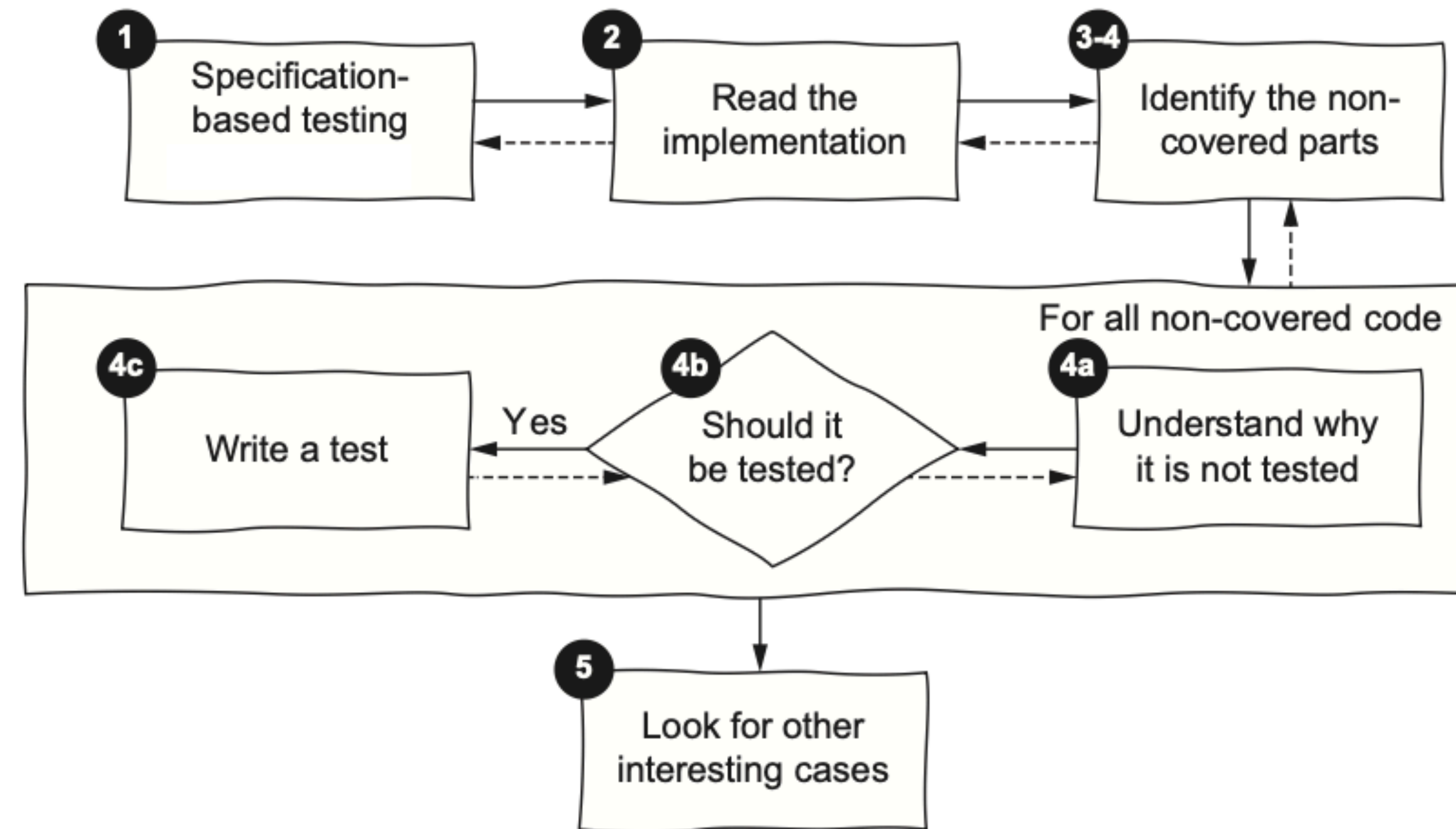
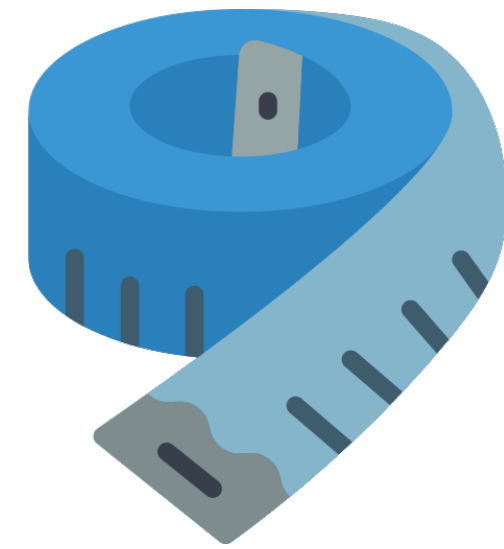
SOFTWARE TESTING - 2025-26

MUTATION ANALYSIS

LESSON 10
14/04/2026
VINCENZO RICCIO

MOTIVATION

WHAT STRUCTURAL COVERAGE TELLS US



- Code coverage tells us which parts of the code were executed
- Coverage is useful to find under-tested areas

100% CODE COVERAGE $\neq$ ABSENCE OF FAULTS

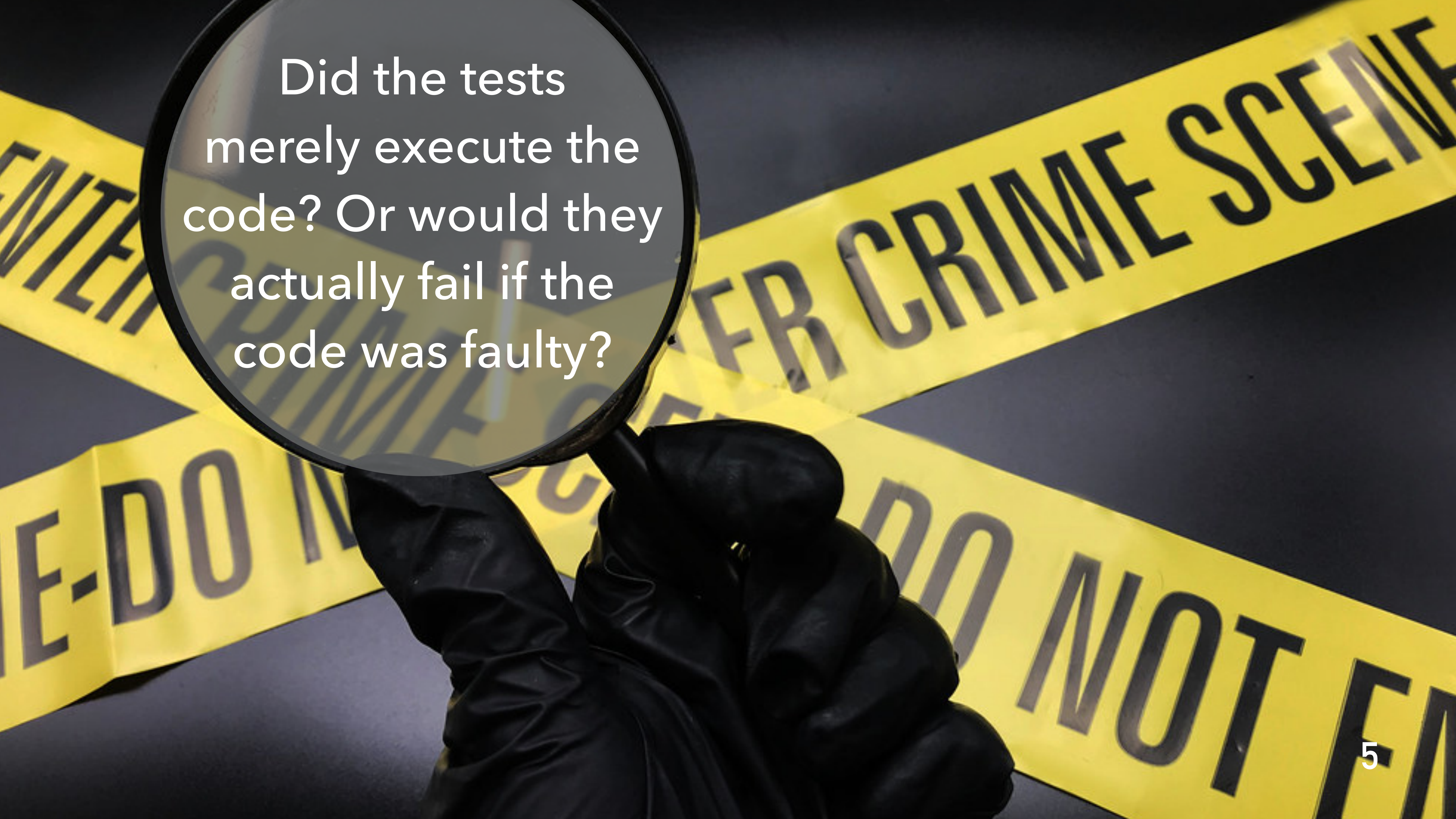
- 100% code coverage does not guarantee the absence of faults: a bug may remain hidden if the relevant input values are never tested
- **Limitation:** Coverage measures **execution**, not fault detection

```
def reciprocal_sign(x: int) -> float:
    if x >= 0:
        return 1 / x          # bug when x == 0
    else:
        return -1.0
```

```
def test_reciprocal_sign():
    assert reciprocal_sign(2) == 0.5
    assert reciprocal_sign(-3) == -1.0
```

```
===== test session starts =====
collecting ... collected 1 item

coverage_bug.py::test_reciprocal_sign PASSED [100%]
```

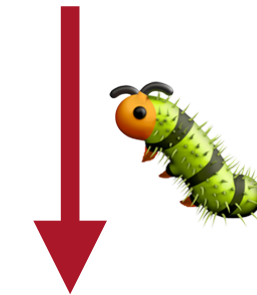
A hand wearing a black nitrile glove holds a magnifying glass over a dark surface. Yellow crime scene tape with the words "DO NOT ENTER CRIME SCENE" is stretched across the background. The magnifying glass is focused on a text overlay.

Did the tests
merely execute the
code? Or would they
actually fail if the
code was faulty?

A SMALL CHANGE, WOULD TESTS NOTICE?

If I slightly break
the program, will
my tests notice?

```
def is_adult(age: int) -> bool:  
    return age >= 18
```



```
def is_adult(age: int) -> bool:  
    return age > 18
```

- Many tests may still pass, code coverage may remain 100%, yet a boundary fault indeed exists → false confidence
- A test may execute a branch without checking the right behavior
- A good test suite will expose this fault

A SMALL CHANGE, WOULD TESTS NOTICE?

If I slightly break
the program, will
my tests notice?

```
def is_adult(age: int) -> bool:  
    return age > 18
```

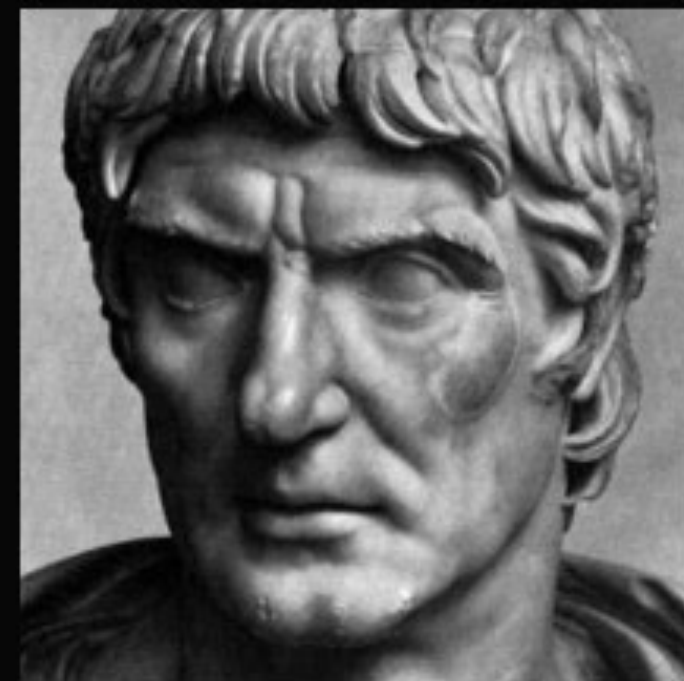
Not in this case!



```
def test_is_adult_for_age_above_18():  
    assert is_adult(20) is True  
  
def test_is_not_adult_for_age_below_18():  
    assert is_adult(10) is False
```

FROM COVERAGE TO FAULT SENSITIVITY

- 💡 **Key idea:** Systematically introducing small changes to the program, then checking whether the tests detect them
- We need to test the tests!
 - **Goal:** Evaluate the fault-detection power of a test suite



Quis custodiet ipsos custodes? -
Who will watch the watchers?

~ Juvenal

THE MAIN IDEA OF MUTATION ANALYSIS

- An ideal test adequacy criterion would evaluate a test suite by its ability to detect faults
- If the faults in a program were known a priori, adequacy could be measured directly in terms of detected faults
- In practice, **real faults are unknown before testing**
- Mutation analysis approximates an ideal fault-based adequacy criterion by injecting artificial faults and checking whether tests detect them 

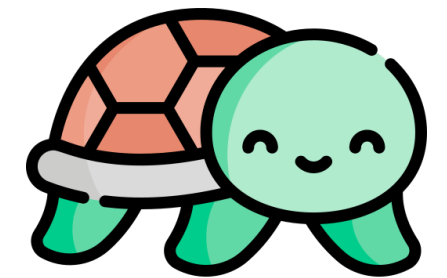
KEY CONCEPTS

MUTANTS

- A mutant is a slightly modified version of the original program under test
- Each mutant contains one small syntactic change meant to simulate a typical programmer mistake

```
def is_adult(age: int) -> bool:  
    return age >= 18
```

Original
program



```
def is_adult(age: int) -> bool:  
    return age > 18
```

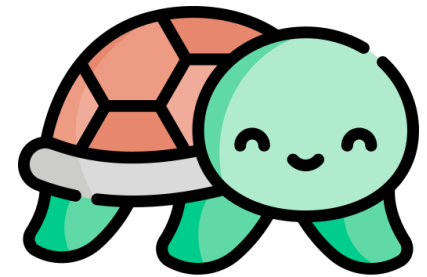
Mutant



KILLED VS SURVIVING MUTANT

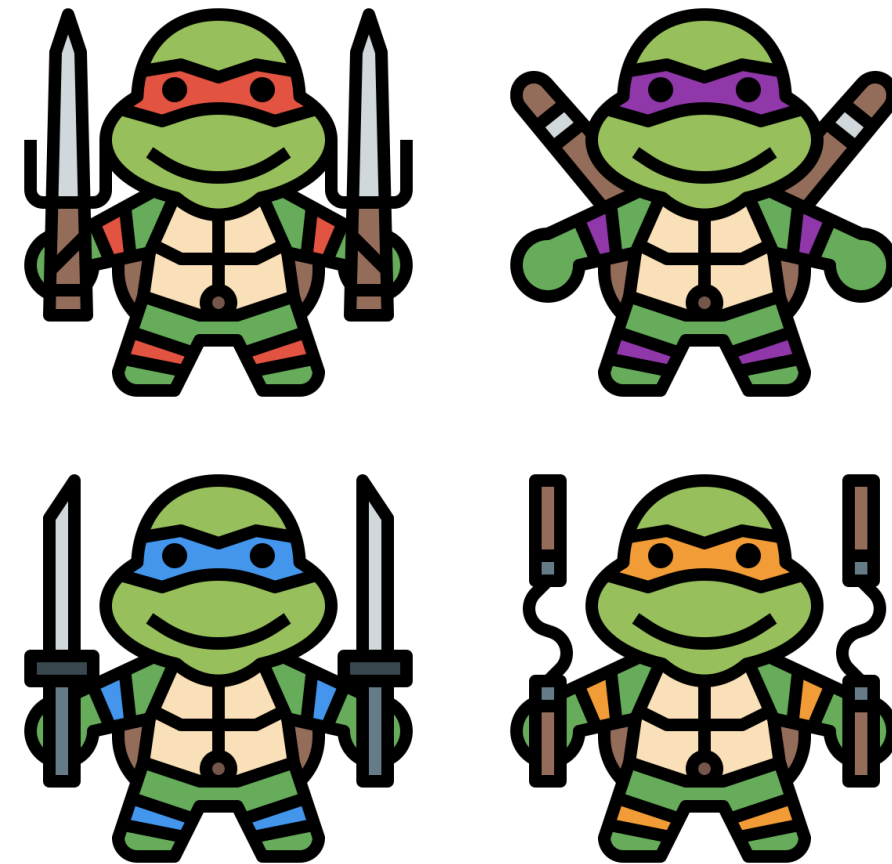
- ▶ A mutant is **killed** when:
 - ⦿ there exists at least one test case whose outcome differs when executed on the original program and on the mutant
 - ⦿ typically, this difference is observed through an assertion, e.g., test passes on the original program but fails on the mutant
 - ⦿ any observable difference counts, e.g., normal termination vs. crash
- ▶ A mutant **survives** a test suite when:
 - ⦿ for every test case in the suite, the outcome is the same whether the test is executed on the original program or on the mutant
 - ⦿ therefore, the test suite does not distinguish the mutant from the original program

MUTATION TESTING

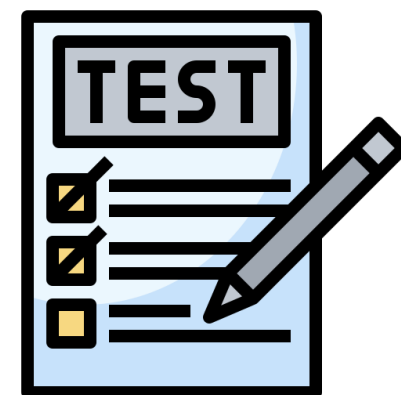


Original
program

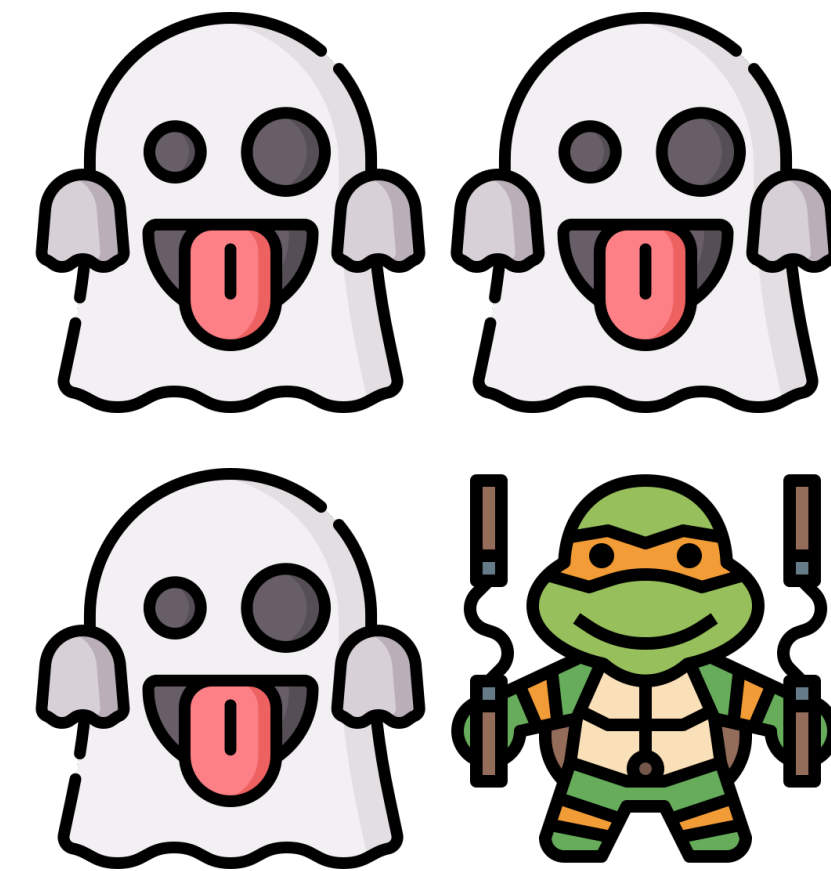
Step 1: Generate many small modified versions (**mutants**) of the program



Mutants



Step 2: Execute the existing test suite against each mutant



Killed
Mutants

Survived
Mutant

Step 3: Check the result

At least one test fails → mutant is **killed**

All tests pass → mutant **survives**

A survived mutant may indicate that the test suite did not detect that type of fault

"KILLING" THE MUTANT

```
def is_adult(age: int) -> bool:  
    return age > 18
```

With the first two
tests the mutant
survives

The third
additional test
"kills" the mutant

```
def test_is_adult_for_age_above_18():  
    assert is_adult(20) is True  
  
def test_is_not_adult_for_age_below_18():  
    assert is_adult(10) is False  
  
def test_is_adult_at_18():  
    assert is_adult(18) is True
```

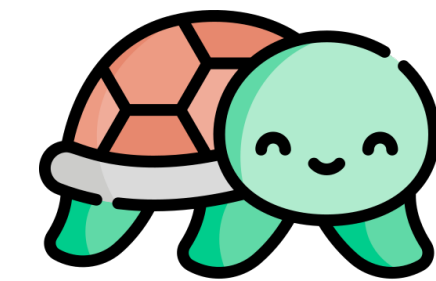
A LIMITATION OF MUTATION TESTING: EQUIVALENT MUTANTS

- Surviving does not always mean detectable fault missed by the test suite, i.e., the test suite is weak
- A mutant may survive because it is **behaviorally identical** to the original program
- In that case, no test case can distinguish the mutant from the original
- Such mutants are called **equivalent mutants**

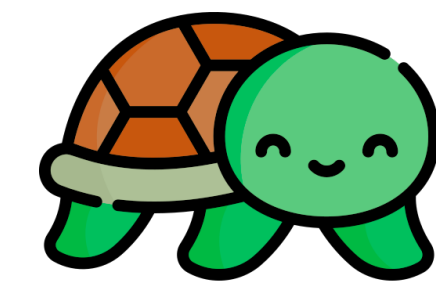


EQUIVALENT MUTANTS

- An equivalent mutant is a syntactically different program that is behaviorally indistinguishable from the original
- A mutant is equivalent if it produces the same observable behavior as the original program for all possible inputs
- No test case can kill equivalent mutants
- Equivalent mutants are problematic because they:
 - ⦿ artificially lower the mutation score
 - ⦿ require human reasoning to identify
 - ⦿ are difficult to detect automatically



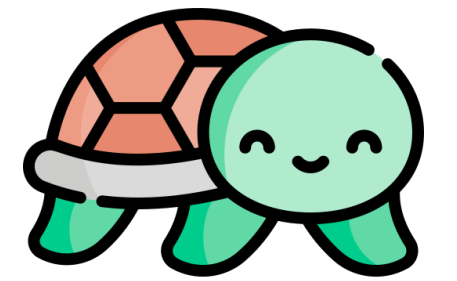
Original
program



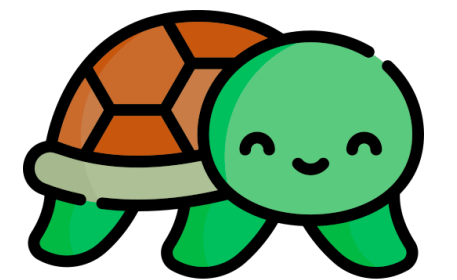
Equivalent
Mutant

EQUIVALENT MUTANTS: EXAMPLE

- Syntax changes but the behavior does not
- The equivalent mutant is syntactically different but for every integer x , it returns the same result
- No test can kill this mutant
- Equivalent mutants must be excluded from the assessment of the test suite (not counted as killable)



```
def add_zero(x: int) -> int:  
    return x + 0
```

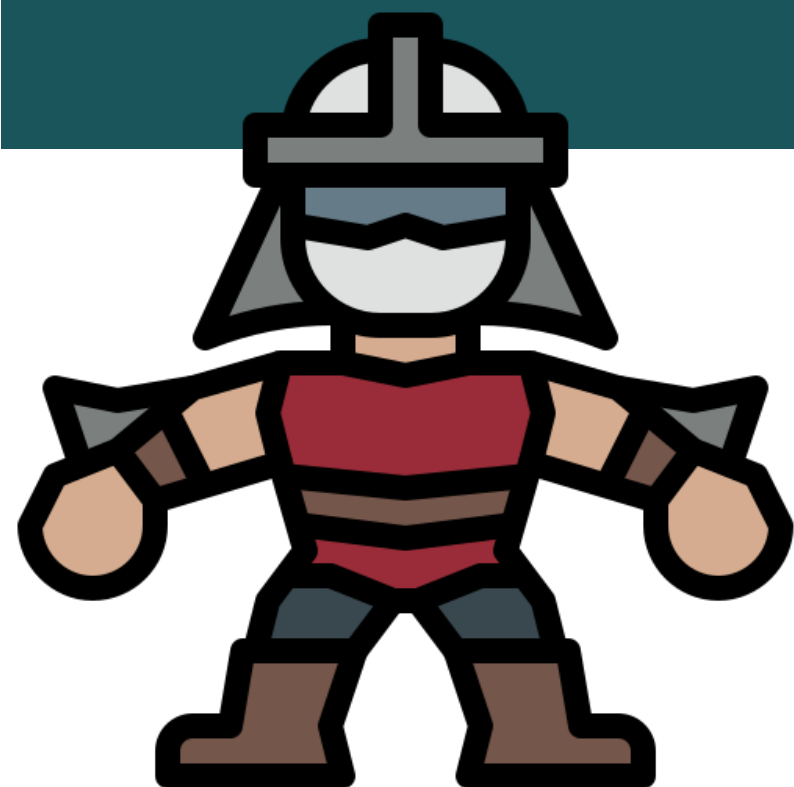


```
def add_zero(x: int) -> int:  
    return x - 0
```

MUTATION SCORE

- Mutation score measures how many non-equivalent mutants are detected by the test suite
- Mutation score is more informative than coverage alone
- Mutation score is not absolute proof of test quality, but it is a complementary, strong adequacy signal

$$\text{Mutation Score} = \frac{\text{killed mutants}}{\text{total mutants} - \text{equivalent mutants}} \times 100$$



20 mutants generated

2 equivalent

15 killed

Mutation Score = $15 / (20 - 2) \times 100 = 83.3\%$

HOW TO INTERPRET MUTATION SCORE

- As an adequacy criterion, mutation score estimates the fault-detection power of a test suite:
 - ◎ **High mutation score:** the tests detect most injected faults, so the suite is likely more effective
 - ◎ **Low mutation score:** many injected faults survive, suggesting weaknesses in the tests
- Mutation score is stronger than coverage alone because it evaluates not only whether code was executed, but whether tests would reveal specific types of faults
- Killed mutants indicate that the test suite is good at detecting the kinds of faults modeled by the corresponding mutation operators

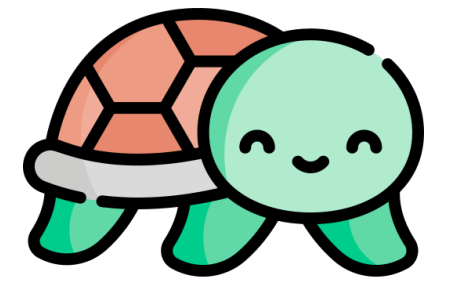
TRIVIAL MUTANTS

- Some mutants are **trivial** because they are killed immediately
 - ◎ Some trivial mutants are detected before execution
 - ◎ Others are killed immediately by the first test
 - ◎ They are **too easy to kill**
- Typical examples are mutants that produce syntax errors, compilation errors, or obvious runtime failures during test execution
- These mutants do not provide useful information about the quality of the test suite
- Therefore, they are usually discarded from mutation analysis



TRIVIAL MUTANTS: EXAMPLE

- The mutant introduces a syntax error
- This mutant is trivial because it is detected immediately: the mutated program is not syntactically valid and cannot be executed



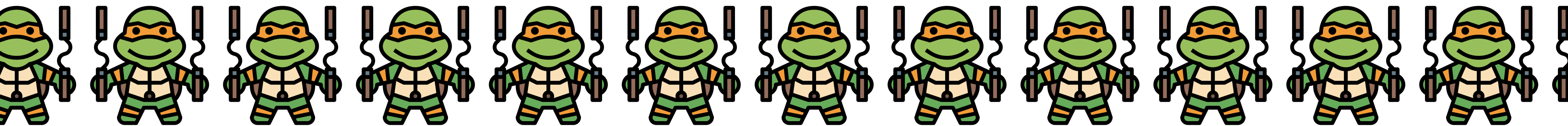
```
def is_even(x: int) -> bool:  
    return x % 2 == 0
```



```
def is_even(x: int) -> bool  
    return x % 2 == 0
```

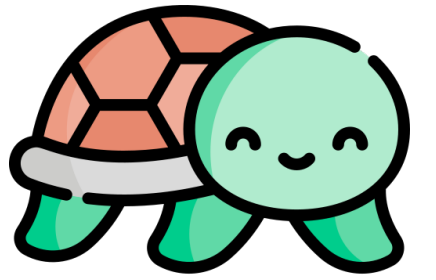
REDUNDANT MUTANTS

- Redundant mutants are different mutants that provide largely the same testing information
- Two or more mutants are **redundant** when they are consistently killed by the same tests
- They exercise essentially the same weakness in the test suite
- Therefore, they contribute little additional evidence about test-suite adequacy
- Reducing redundancy can improve the efficiency of mutation analysis



REDUNDANT MUTANTS: EXAMPLE

- ▶ Both mutants alter the condition in a very similar way
- ▶ A boundary test $x = 0$ is likely to distinguish both from the original
- ▶ So the same test may kill both mutants
- ▶ In practice, one mutant may add little information beyond the other



```
def is_positive(x: int) -> bool:  
    return x > 0
```



```
def is_positive(x: int) -> bool:  
    return x >= 0
```



```
def is_positive(x: int) -> bool:  
    return not (x <= 0)
```

FROM FAULT TAXONOMY TO MUTATION OPERATORS

Mutation score
depends on the
mutants we
generate



Family: Rosaceae:
Genus: Potentilla
Species: P. alba



Classification Notes:

- Leaf Shape: Compound
- Flower: White; 5 Parts
- Root: Fibrous

Habitat: Meadow & Woodland



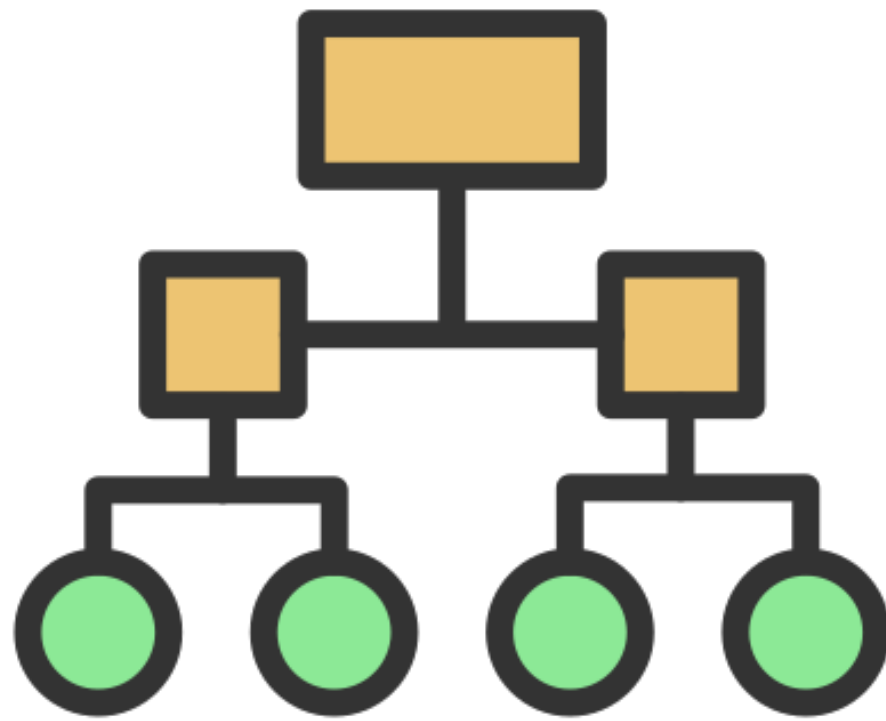
We need to
design
representative
mutations

WHY DO WE NEED MUTATION OPERATORS?

- Mutation testing does not introduce arbitrary changes
- It uses predefined **mutation operators**
- A mutation operator applies a small syntactic change to a supposedly correct program, thereby injecting a fault and producing a mutant
- Each operator simulates a recurring class of programming mistakes
- Mutation operators are grounded in fault taxonomies

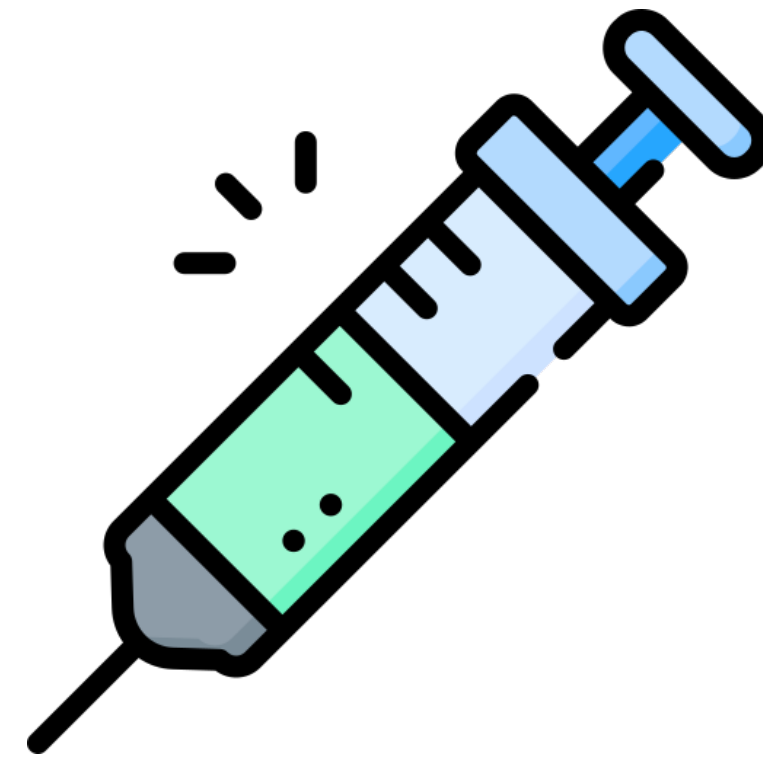


MUTATION TESTING



Step 1: Build a fault taxonomy

Collect recurring programmer mistakes and group them into fault categories



Step 2: Derive mutation operators

For each fault category, define small syntactic transformations that model that kind of mistake



Step 3: Generate mutants automatically

Apply mutation operators to the original program to create faulty variants that can be executed against the test suite

COMMON CLASSES OF PROGRAM FAULTS

Fault Class	Description
Relational faults	wrong comparison operator, boundary mistakes
Arithmetic faults	wrong arithmetic operator, sign mistakes
Logical faults	wrong boolean connective, missing negation
Constant/boundary value faults	wrong literal, off-by-one values

FROM FAULT CLASSES TO MUTATION OPERATORS

Fault Class	Operator Class	Example Transformation
Relational faults	Relational Operator Replacement	$> \rightarrow \geq, < \rightarrow \leq, == \rightarrow !=$
Arithmetic faults	Arithmetic Operator Replacement	$+ \rightarrow -, * \rightarrow //$
Logical faults	Logical Connector Replacement	and $\rightarrow$ or , or $\rightarrow$ and
	Negation Insertion/Deletion	Insert/remove not
Constant/ boundary value	Constant Replacement	$0 \rightarrow 1, n \rightarrow n+1$

WHAT KINDS OF WEAKNESSES DO DIFFERENT MUTANTS EXPOSE?

- Different surviving mutants often reveal the following weaknesses in test design

Mutant family	Weakness exposed in the test suite
Relational mutants	Missing boundary tests
Arithmetic mutants	Missing value-based assertions on computed results
Logical mutants	Missing boolean scenario tests covering decision combinations
Negation mutants	Missing tests for the opposite truth condition
Constant/boundary-value mutants	missing off-by-one and special-value tests

WHY DO WE APPLY ONLY ONE MUTATION AT A TIME?

- Each mutant usually contains only **one small change**
- This makes it easier to relate the mutant to one specific fault
- If multiple mutations were applied at the same time, it would be harder to understand:
 - ⦿ which change caused the tests to fail
 - ⦿ which weakness in the test suite was exposed
- Using one mutation at a time keeps the analysis:
 - ⦿ simpler
 - ⦿ more diagnostic, as one mutation may hide the effect of another
 - ⦿ easier to automate and interpret

HOW OFTEN SHOULD DIFFERENT MUTATION OPERATORS BE APPLIED?

- Choosing the mutation operators is not enough. We must also decide how often to use each one
-  **Statistical approach**
 - ◎ observe the distribution of real faults in software systems
 - ◎ apply mutation operators in similar proportions
-  **Fair approach**
 - ◎ count how many times each type of mutation could be applied in the program
 - ◎ inject mutations proportionally to these opportunities

LIMITATIONS

LIMITATIONS: MODELING REALISTIC FAULTS

- The effectiveness of mutation analysis depends on how well the generated mutants represent real faults
- Defining good mutation operators is not easy
- Mutation operators are usually designed:
 - ◎ from researchers' and practitioners' experience about common programming mistakes
 - ◎ from empirical or statistical analyses of faults observed in real software



LIMITATIONS: COST

- Mutation analysis may generate a large number of mutants
- Each mutant must be analyzed by running the test suite against it
- As a result, mutation analysis may require many repeated test executions
- This can make mutation analysis computationally expensive, especially for large systems or slow test suites
- Therefore, the practical use of mutation analysis often requires balancing:
 - accuracy
 - time
 - computational resources



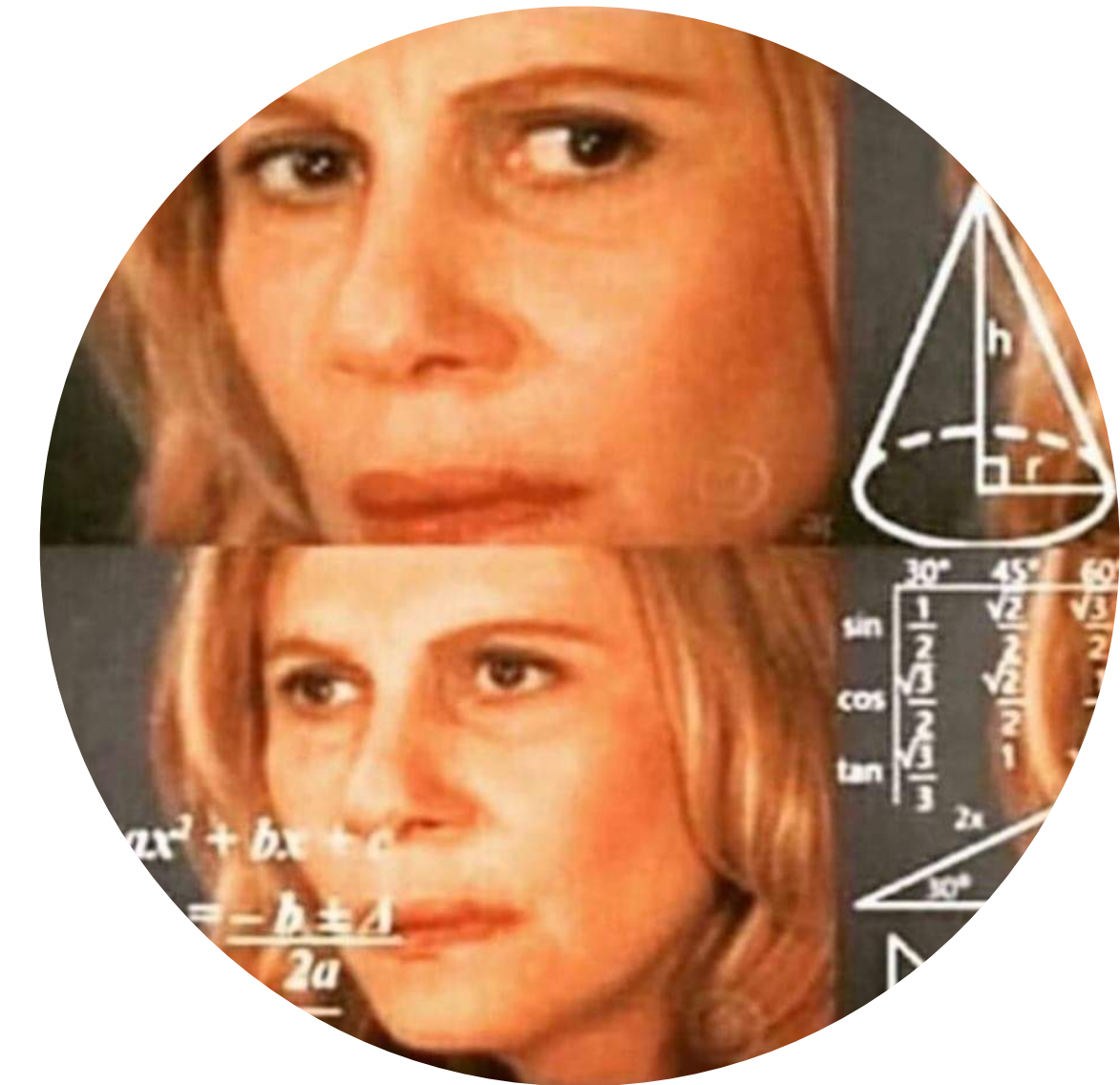
GENERIC OPERATORS: STRENGTH AND WEAKNESS

- ▶ Most mutation operators are intentionally generic
- ▶ This makes them applicable across different programs, domains, and languages
- ▶ Since they are generic, these operators support automatic mutant generation
- ▶ However, generic operators may miss faults that depend on the specific logic of the application (e.g., semantic faults)



COMPLEX FAULTS ARE HARDER TO MODEL

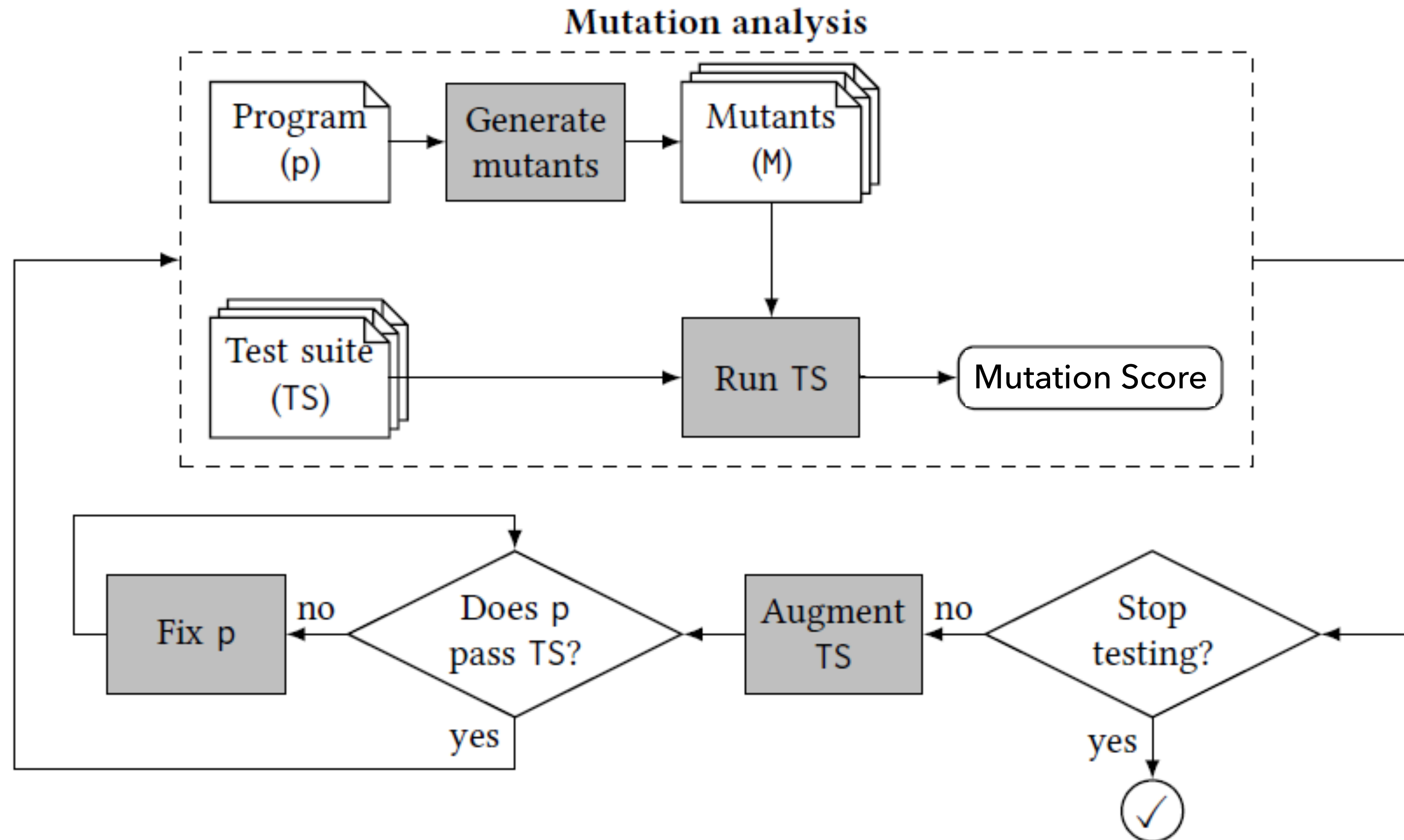
- Some faults are simple and local, so they can be injected automatically
- Other faults are more complex and depend on the semantics of the application
 - incorrect business rules
 - missing special cases
- These faults are difficult to represent with generic mutation operators
- In many cases, they require manual modeling and remain meaningful only for that specific application



MUTATION ANALYSIS IN PRACTICE

MUTATION BASED TESTING

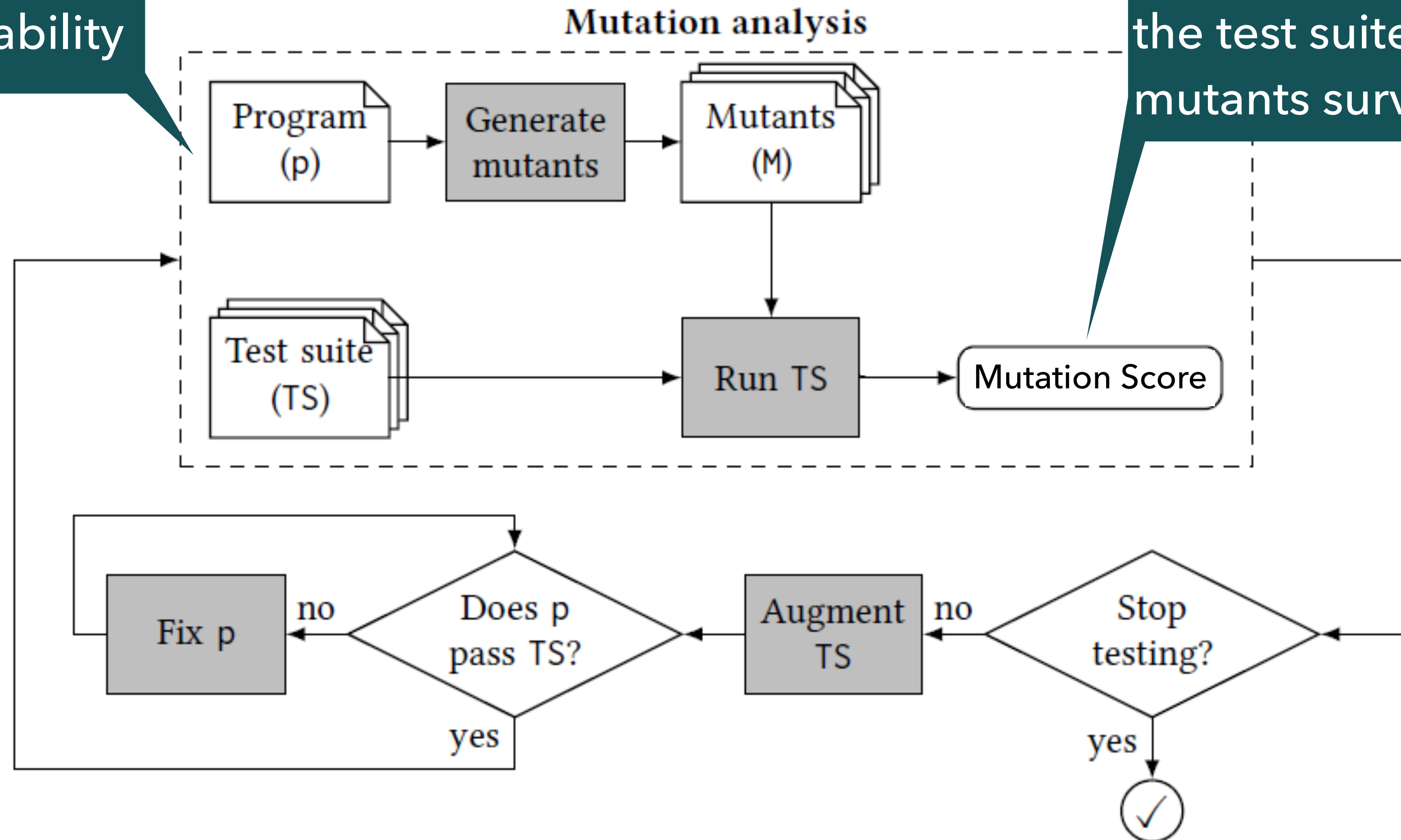
MUTATION ANALYSIS AS AN ITERATIVE TEST IMPROVEMENT PROCESS



STEP 1: RUN MUTATION ANALYSIS

Use the current test suite and evaluate its fault detection ability

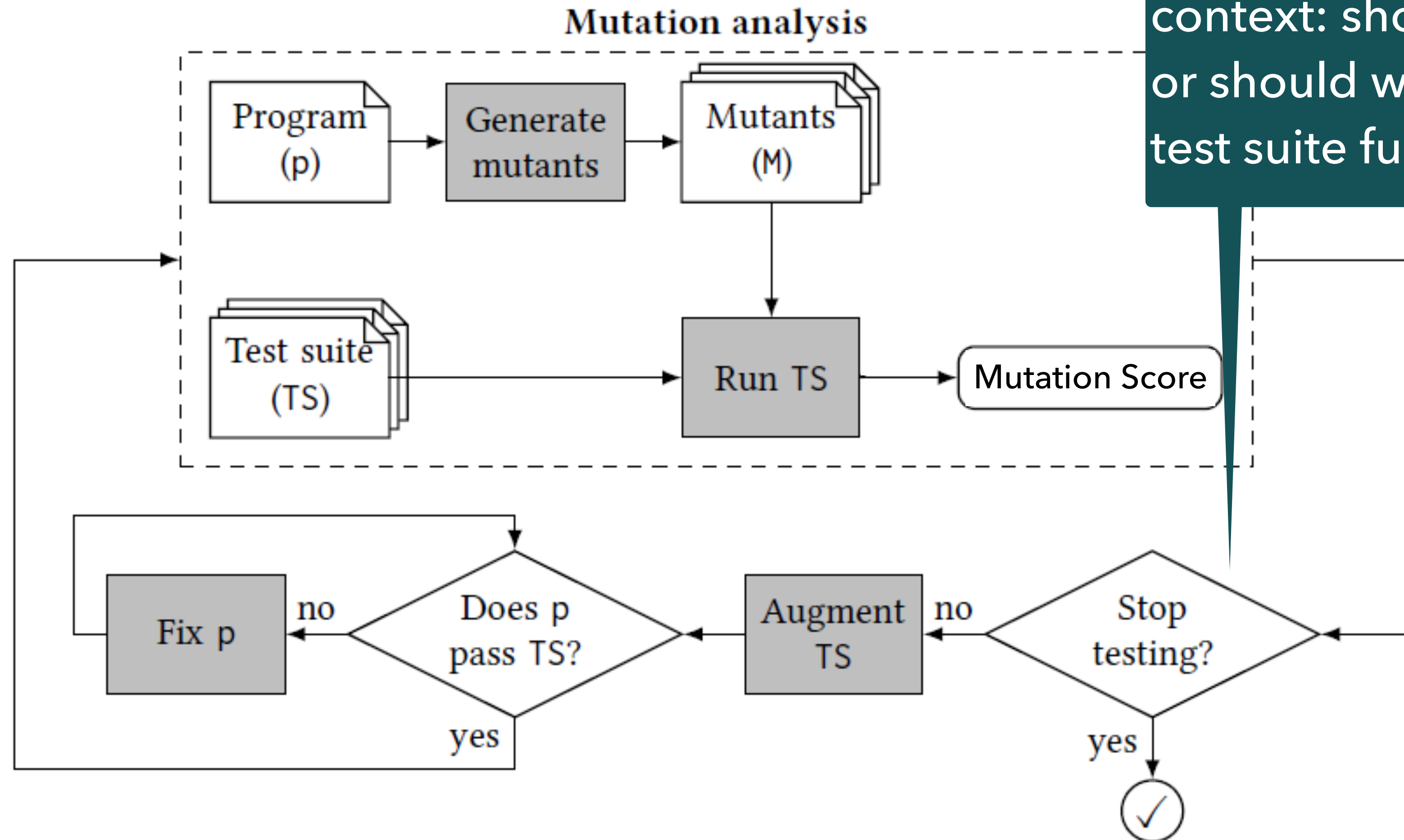
Mutation score is not the final goal but a tool to analyze the adequacy of the test suite and which mutants survive



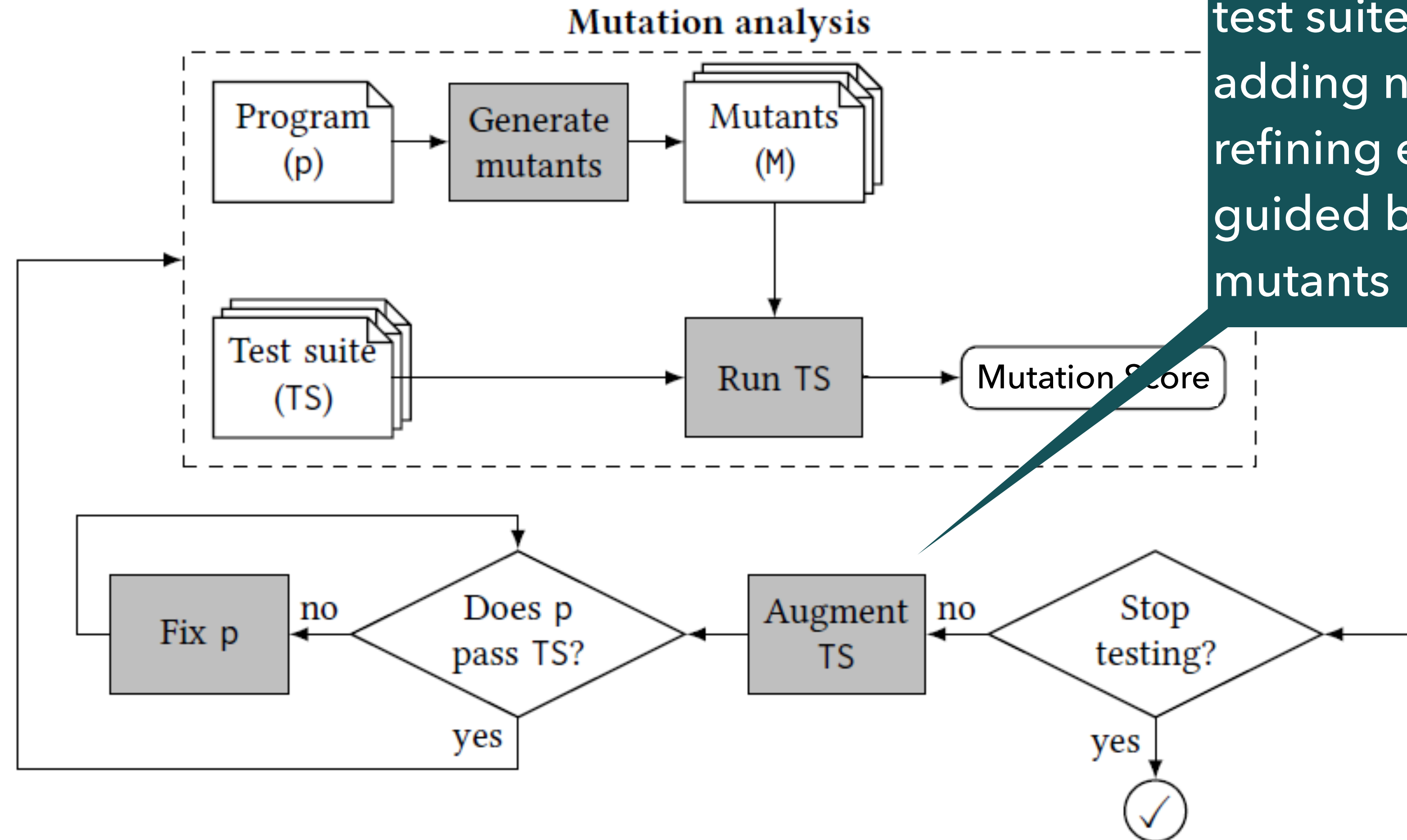
STEP 2: DECIDE WHETHER THE TESTS ARE GOOD ENOUGH



We evaluate if the current mutation score is acceptable for our context: should we stop, or should we improve the test suite further?



STEP 3: TEST SUITE AUGMENTATION

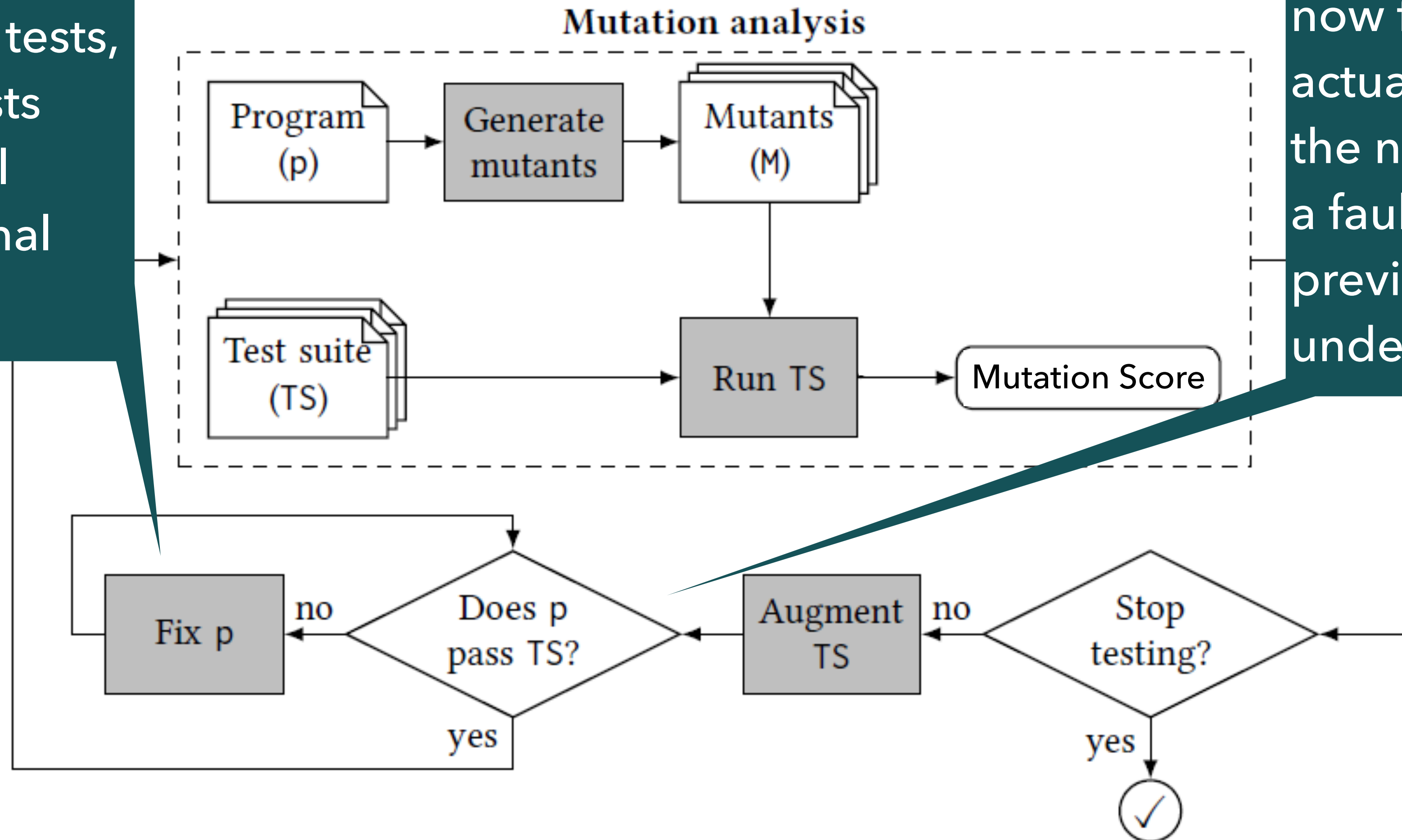


If the current mutation score is not acceptable, we must augment the test suite. That means adding new tests or refining existing ones, guided by surviving mutants

STEP 4: RE-RUN THE STRONGER TESTS ON THE ORIGINAL PROGRAM

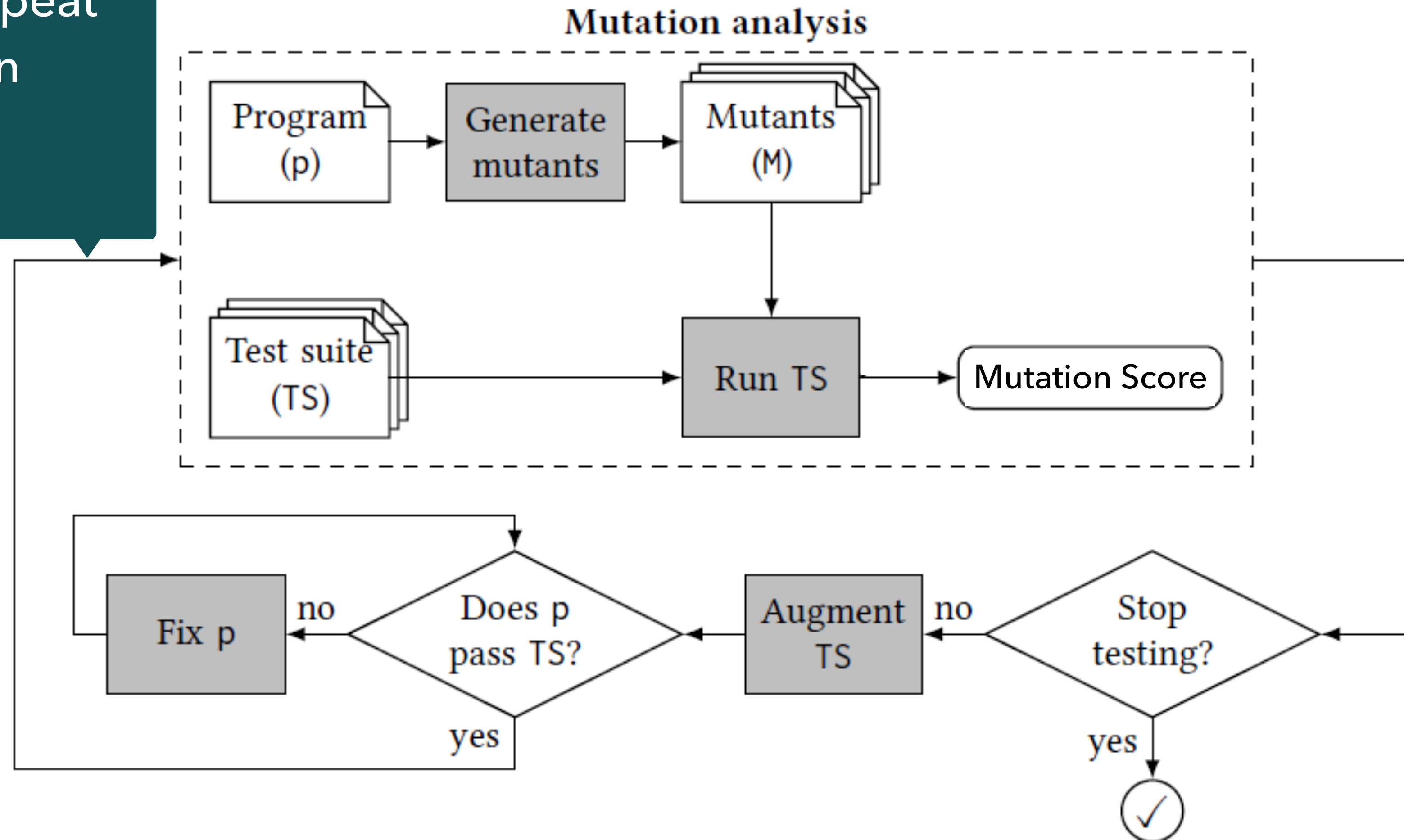
Mutation analysis helps us improve tests, and improved tests may in turn reveal faults in the original program

When we strengthen the test suite, the original program may now fail. That is actually good news: the new tests exposed a fault that had previously gone undetected



STEP 5: ITERATE

Mutation analysis supports iterative improvement: repeat until the mutation adequacy is satisfactory



Mutation analysis does not directly fix the program

It helps us discover weaknesses in the test suite

Then stronger tests may reveal actual faults in the original program



WHAT IS **mutmut** ?

- **mutmut** is a mutation testing tool for Python
- Automatically creates mutants by applying small changes to the source code
- It runs the existing pytest test suite against mutants
- It reports which mutants are killed and which survive
- **Advantages:**
 - ◎ **Open source:** publicly available and community-maintained
 - ◎ **Python-focused:** designed for mutation testing with Python projects
 - ◎ **Simple CLI/TUI:** supports browsing and applying mutants interactively

SIMPLE MUTMUT SETUP

A stable pytest setup comes first; mutmut is then added on top to automate mutation analysis

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
pip install mutmut pytest
pytest
mutmut run
```

By default MutMut uses pytest and looks for pytest tests in tests/ or test/

TROUBLESHOOTING DISCLAIMER

- ▶ Demo stack: Python 3.12 + pytest 8.4.2 + mutmut 3.5.0 on macOS
- ▶ On macOS, mutmut has known fork-related issues and may still show segfaults in some (e.g., **my**) environments
- ▶ Python 3.14 has reported segfault issues with mutmut, so I avoid it for the demo.
- ▶ If something fails, it may be a tool version/platform issue rather than an issue with you tests or program

```
(.venv) vincenzoriccio@Vincenzos-MacBook-Pro coverage-lab % mutmut results
max_consecutive.x_max_consecutive_items__mutmut_1: segfault
max_consecutive.x_max_consecutive_items__mutmut_2: segfault
max_consecutive.x_max_consecutive_items__mutmut_3: segfault
max_consecutive.x_max_consecutive_items__mutmut_4: segfault
max_consecutive.x_max_consecutive_items__mutmut_5: segfault
max_consecutive.x_max_consecutive_items__mutmut_6: segfault
max_consecutive.x_max_consecutive_items__mutmut_7: segfault
max_consecutive.x_max_consecutive_items__mutmut_8: segfault
max_consecutive.x_max_consecutive_items__mutmut_9: segfault
max_consecutive.x_max_consecutive_items__mutmut_10: segfault
max_consecutive.x_max_consecutive_items__mutmut_11: segfault
max_consecutive.x_max_consecutive_items__mutmut_12: segfault
max_consecutive.x_max_consecutive_items__mutmut_13: segfault
max_consecutive.x_max_consecutive_items__mutmut_14: segfault
max_consecutive.x_max_consecutive_items__mutmut_15: segfault
max_consecutive.x_max_consecutive_items__mutmut_16: segfault
max_consecutive.x_max_consecutive_items__mutmut_17: segfault
max_consecutive.x_max_consecutive_items__mutmut_18: segfault
max_consecutive.x_max_consecutive_items__mutmut_19: segfault
max_consecutive.x_max_consecutive_items__mutmut_20: segfault
max_consecutive.x_max_consecutive_items__mutmut_21: segfault
```



TEST SUITE ASSESSMENT WITH MUTMUT

We use mutmut to
check whether our
test suite can
detect injected
faults

```
def is_adult(age: int) -> bool:  
    return age >= 18
```

```
def test_is_adult_for_age_above_18():  
    assert is_adult(20) is True  
  
def test_is_not_adult_for_age_below_18():  
    assert is_adult(10) is False
```

TEST SUITE ASSESSMENT WITH MUTMUT: SURVIVING MUTANTS

`mutmut run` generates mutants and runs our tests on them to check if they are killed or survive

```
(.venv) vincenzoriccio@Vincenzos-MacBook-Pro mutmut_playground % mutmut run
:: Generating mutants
  done in 35ms (1 files mutated, 0 ignored, 0 unmodified)
: Running stats
  done
:: Running clean tests
  done
: Running forced fail test
  done
Running mutation testing
:: 21/21 🎉 19 😊 0 🕒 0 🤔 0 😞 2 🚫 0 🧙 0
238.00 mutations/second
```

TEST SUITE ASSESSMENT WITH MUTMUT: INSPECT SURVIVING MUTANTS

A surviving mutant is not the end of the analysis. It is the start of an investigation 🕵️:

1. identify the changed behavior in the survived mutant
2. ask what input distinguishes mutant from original, i.e., for which they return different outcomes
3. If you cannot find any input that can distinguish mutant from original, the mutant is considered equivalent (**note: it is not easy to prove equivalence in practice**)
4. derive the missing test to kill the mutant (if not equivalent)

TEST SUITE ASSESSMENT WITH MUTMUT: INSPECTING THE WEAKNESSES

We use `mutmut browse` to explore the survived mutants

- Two surviving mutants: `return age >= 19`, `return age > 18`
- Both mutants change the adult threshold
- In the original function, `age = 18` returns `True`
- In both surviving mutants, `age = 18` returns `False` -> changes behavior -> **not equivalent**
- Since the current tests do not check `age = 18`, both mutants survive
- Bottom line: `mutmut` reveals a missing

Path	🟡	🟠	🔴	🟢	🟣	name	status
src/example1.py	2	0	0	0	0	example1.x_is_adult__mutmut_1	🟡
						example1.x_is_adult__mutmut_2	🟡

```

Survived (exit_code=0): No test detected this mutant. (press t to view tests
executed for this mutant)

--- src/example1.py
+++ src/example1.py
@@ -1,2 +1,2 @@
def is_adult(age: int) -> bool:
-   return age >= 18
+   return age > 18
q Quit  r Retest mutant  f Retest function  m Retest module  a Apply mutan  ^p pal

```

TEST SUITE ASSESSMENT WITH MUTMUT: WHAT TEST TO ADD

We add an additional boundary test that should kill both surviving mutants 

```
def is_adult(age: int) -> bool:  
    return age >= 18
```

```
def test_is_adult_for_age_above_18():  
    assert is_adult(20) is True  
  
def test_is_not_adult_for_age_below_18():  
    assert is_adult(10) is False  
  
def test_is_adult_at_18():  
    assert is_adult(18) is True
```

TEST SUITE ASSESSMENT WITH MUTMUT

We reassess our tests with `mutmut run` : now all the mutants are successfully killed

```
(.venv) vincenzoriccio@Vincenzos-MacBook-Pro mutmut_example % mutmut run
.: Generating mutants
  done in 11ms (0 files mutated, 0 ignored, 1 unmodified)
.: Listing all tests
.: Running clean tests
  done
.: Running forced fail test
  done
Running mutation testing
.: 2/2 🎉 2 😐 0 🕒 0 🤔 0 😞 0 🚫 0 🧙 0
119.85 mutations/second
```

REFERENCES



REFERENCES

- Ammann, Offutt, Introduction to Software Testing, Chapter 9
- Aniche, Maurício. Effective Software Testing: A developer's guide. Simon and Schuster, 2022, Chapter 3.11
- Tramontana, P., Material from Software Testing course